



Minimized Input Optimization API User Guide

Software Version: 1.0

March 2022

Notice

This document contains information that is proprietary to Sapiens International Corporation N.V or any of its affiliates. No part of this publication may be reproduced, modified, or distributed without prior written authorization of Sapiens International Corporation N.V or any of its affiliates.

This document is provided as is, without warranty of any kind.

Trademarks

Sapiens International Corporation N.V.® is a registered trademark.

Sapiens Decision™ is a trademark of Sapiens International Corporation N.V.

Other names mentioned in this publication are owned by their respective holders.

Statement of Conditions

The information contained in this document is subject to change without notice. Sapiens International Corporation N.V. and/or any of its affiliates shall not be liable for errors contained herein or for any damage or losses arising out of or in connection with the furnishing, performance, or use of this document or the software supplied with it.

Caution

Any changes or modifications of the software not expressly approved in writing by the manufacturer could void the user's authority to use the application and its warranty.

Document Revision History

Revision	Release Date	Author	Description	Approved by
1.0	March 2022	Decision Documentation	Describes MIO adapter and how to work with it	Decision Product Management

Document Authorization

This document is digitally signed and authorized by the Sapiens Decision documentation team. The steps to authenticate the digital certificate are documented in the Index file present in the documentation package. In case of any queries or feedback on the document, send an e-mail to Decision_documentation@sapiens.com.



Copyright © 2022 by Sapiens International Corporation N.V. All rights reserved.

Contents

1. Introducing Minimized Input Optimization	4
2. Environment Configuration	5
2.1 DE.....	5
2.1.1 Enabling the MIO API for use with DE	5
2.1.2 Configuring an environment to use the MIO API with DE.....	5
2.2 Java	6
2.2.1 Enabling the MIO Java Format Adapter	6
2.2.2 Configuring an environment to use the MIO Java Format Adapter	6
3. Sample Requests and Responses	7
3.1 DE.....	7
3.2 Java	8
4. Algorithm.....	8
5. Known Limitations.....	12

1. Introducing Minimized Input Optimization

The input required to run a decision often comes from interaction with a user via a user interface. As an example, think of filing a property insurance claim. Typically, this requires the completion of a questionnaire describing the type of damage, what was damaged, the extent of the damage, etc. Some of the questions may depend on the responses to earlier questions, such that, for example, details about repair estimates will not be requested if an estimate was not done. The scripting of the questionnaire and the logic behind the questionnaire is commonly programmed separately from the business logic that is dependent on the input from the questionnaire. Since the script logic is in fact driven by the underlying business logic, this means that the same logic appears in both places. Any change to the inputs required by the business logic would also mean a change to the scripting logic.

Instead of manually creating the scripting logic, the Minimized Input Optimization (MIO) API provides an API for the user interface to request the next question(s) to be displayed to the user. The MIO API selects which question(s) to ask next to provide the optimal user experience, that is, ask the user the fewest possible number of questions. It does this by examining the declarative business logic expressed in a Decision View and determining which question(s), given particular responses to each, would yield a Decision conclusion. If the responses received are not the ones that yield a conclusion right away, the process iterates using all the received inputs from previous iterations to determine the next set of questions to ask.

The MIO API enables automated UI generation. Instead of scripting the questionnaire logic, the user interface would dynamically invoke the MIO API and based on the returned response, display a set of questions to the user. At each iteration, the MIO API returns either the Decision View conclusion or the set of Fact Types required next. The user interface would then display the required Fact Types as questions and answers to the user (converting the Fact Type name to question/answer text in the user's language through a lookup table or other similar means) and formatting the possible answers as appropriate for the Fact Type data type (e.g., using a radio button for Indicator Fact Types). As an alternate design option, the user interface could display all possible questions to the user and highlight the ones it recommends answering next.

The MIO API is a good fit for any use case where decision input comes from a UI and there is a need to optimize the UI.

Sample use cases:

- Insurance claims
- Know Your Customer (KYC)
- Product selection
- Loan eligibility

The MIO API is supported with Decision Execution (DE) and the Java Adapter. When used with a Decision Flow, the algorithm attempts to process all the Decision Tasks within the Decision Flow until it encounters a gateway where it cannot determine the branch to be taken or the end of

the Decision Flow. The MIO API is then applied to each Decision View independently. The required Fact Types to request next are provided for each processed Decision View independently and available to be consolidated by the calling application.

2. Environment Configuration

2.1 DE

Configuring the MIO API for use with DE requires:

- Enabling the MIO API for use with DE
- Configuring an environment to use the MIO API with DE

2.1.1 Enabling the MIO API for use with DE

Add the following property to the “bdms.cust.properties” file:

Property	Possible Settings
<i>mio.de.enabled=true</i>	To enable the MIO API for use with DE. Note: By default, the configuration value is set as false .

2.1.2 Configuring an environment to use the MIO API with DE

1. Log in as a Community Administrator.
2. In **Administration** → **Communities** → **Environment Management**, define a new environment.
3. Select **DECISION Execution (DE)** as the Format.
4. Select **DE (MIO)** for the Version of the Format.

The screenshot shows the Sapiens Decision Environment Management interface. The top navigation bar includes 'Home', 'Tasks (41)', 'Repository', 'Reports', and 'Administration'. The breadcrumb trail is 'Communities > WORLD > Environment Management > New Environment'. The form contains the following fields:

- Name ***: Decision Execution + MIO
- Description**: (empty text area)
- Type ***: Production (dropdown)
- Format**: DECISION Execution (DE) (dropdown, highlighted with a red box)
- Version ***: DE (MIO) (dropdown, highlighted with a red box)

Below the form, there is a table with columns 'NAME' and 'VALUE'. The table contains one row: 'Decimal Data Type for the Java Adapter (Rel...' with the value 'BigDecimal'.

5. In the **Repository** group, enter the URL for the deployment.

6. Save the environment.

2.2 Java

Configuring the MIO API for use with the Java Adapter requires:

- Enabling the MIO Java Format Adapter
- Configuring an environment to use the MIO Java Format Adapter

2.2.1 Enabling the MIO Java Format Adapter

Add the following property to the “bdms.cust.properties” file:

Property	Possible Settings
<i>mio.java.enabled=true</i>	To enable the MIO Java Format Adapter. Note: By default, the configuration value is set as false .

2.2.2 Configuring an environment to use the MIO Java Format Adapter

1. Log in as a Community Administrator.
2. In **Administration** → **Communities** → **Environment Management**, define a new environment.
3. Select **MIO Java** as the Format.

The screenshot shows the Sapiens Decision web interface. The top navigation bar includes 'Home', 'Tasks (41)', 'Repository', 'Reports', and 'Administration'. The breadcrumb trail is 'Communities > WORLD > Environment Management > New Environment'. The form contains the following fields:

- Name ***: Text input with 'Java + MIO'.
- Description**: Text area.
- Type ***: Dropdown menu with 'Production' selected.
- Format**: Dropdown menu with 'MIO Java' selected. This field is highlighted with a red rectangular box.
- Parameters Table**:

NAME	VALUE
Decimal Data Type *	BigDecimal
Decision Input Bean Generator *	noInputBeanGenerator
Decision Input Bean Package Name *	

4. Fill out the Format parameters as required.
5. In the **Repository** group, enter the URL for the deployment.
6. Save the environment.

3. Sample Requests and Responses

3.1 DE

The MIO API is supported in DE with RESTful and SOAP web services input request messages. To use the MIO API with DE, deploy a revision to the environment defined in the section, [Configuring an environment to use the MIO API with DE](#).

Send a REST or SOAP message with some or all the input Fact Types omitted. Note the empty Root group in the sample input message for a Decision View below:

```
{
  "executableKey": {
    "artifactKey": {
      "releaseName": "UnderwritingProject",
      "tagName": "Tag1",
      "name": "PolicyRenewalMethod",
      "artifactType": "DECISION"
    },
    "version": "1.0",
    "viewName": "Base"
  },
  "executionInput": {
    "Root": {
    }
  }
}
```

Sample initial input message

The resulting output may include a requiredFactTypes element as highlighted below:

```
{
  "executedVersion": "1.0",
  "requiredFactTypes": ["PolicyPricingTier"],
  "name": "Determine Policy Renewal Method (View: Base)",
  "executedViewName": "Base",
  "executableKey": {
    "artifactKey": {
      "releaseName": "UnderwritingProject",
      "tagName": "Tag1",
      "name": "PolicyRenewalMethod",
      "artifactType": "DECISION"
    },
    "effectiveDate": null,
    "viewName": "Base",
    "version": "1.0"
  },
  "profileStopWatch": [
    {
      "operation": "Execute DV (Execution Manager)",
      "elapsedTime": 0
    }
  ]
}
```

Output for the sample initial input message

For the output above, the next Fact Type (question) to be provided is PolicyPricingTier, which, with an appropriate value, can return a final conclusion for the next request. The next input message should include this Fact Type but may also include any other input Fact Types. Submitting this message will return different required Fact Types (could be more than one) in the requiredFactTypes element. You would then add input values for these Fact Types into the next input message and submit again. Proceed in this way, until a conclusion is reached.

3.2 Java

To use the MIO API with the Java Adapter, deploy a revision to the environment defined in the section, [Configuring an environment to use the MIO Java Format Adapter](#).

You must create some Java code such as the sample snippet below to execute the code generated by the Java Adapter. The snippet shows instantiation and execution of a decision without passing any input data. After executing the decision, it uses the `{decision}.getRequiredFactNames()` method to retrieve and then print the collection of Fact Types (questions) that are required next. If the `{decision}.getRequiredFactNames()` returns an empty value then, the Decision View will be complete and you can obtain the Decision View conclusion as shown below:

```
Collection<String> requiredFtNames, messages;

PolicyRenewalMethod policyRenewalMethod = new PolicyRenewalMethod();
policyRenewalMethod.execute();

requiredFtNames = policyRenewalMethod.getRequiredFactNames();
if (requiredFtNames.isEmpty()) {
    String conclusion = policyRenewalMethod.getConclusion();
    System.out.println(conclusion);

    messages = policyRenewalMethod.getConclusionMessagesCollection("Informational");
    messages.forEach(System.out::println);
} else {
    requiredFtNames.forEach(System.out::println);
}
```

Sample Java code for execution with the MIO API

You must get input values for the required Fact Types, call the `clear()` method, and then re-execute the decision to get the next set of required Fact Types. Proceed in this way, until a conclusion is reached.

4. Algorithm

The general method is to establish a questionnaire per DV, where the questions on the questionnaire are derived from the interim FTs and to develop an execution mode where, upon each answer supplied by the user, you can test whether a specific RFV is determinative, and if it is, whether that causes the DV to become determinative.

Each persistent FT is assumed to be represented by a question on that page (the questions could be contained, e.g., in the model mappings). The questions may appear based on the DV, multiple or one at a time. If the processing of an RFV is not complete, and an additional RFV is

required, then the questions for that RFV is added to the existing page. These are implementation details to be considered.

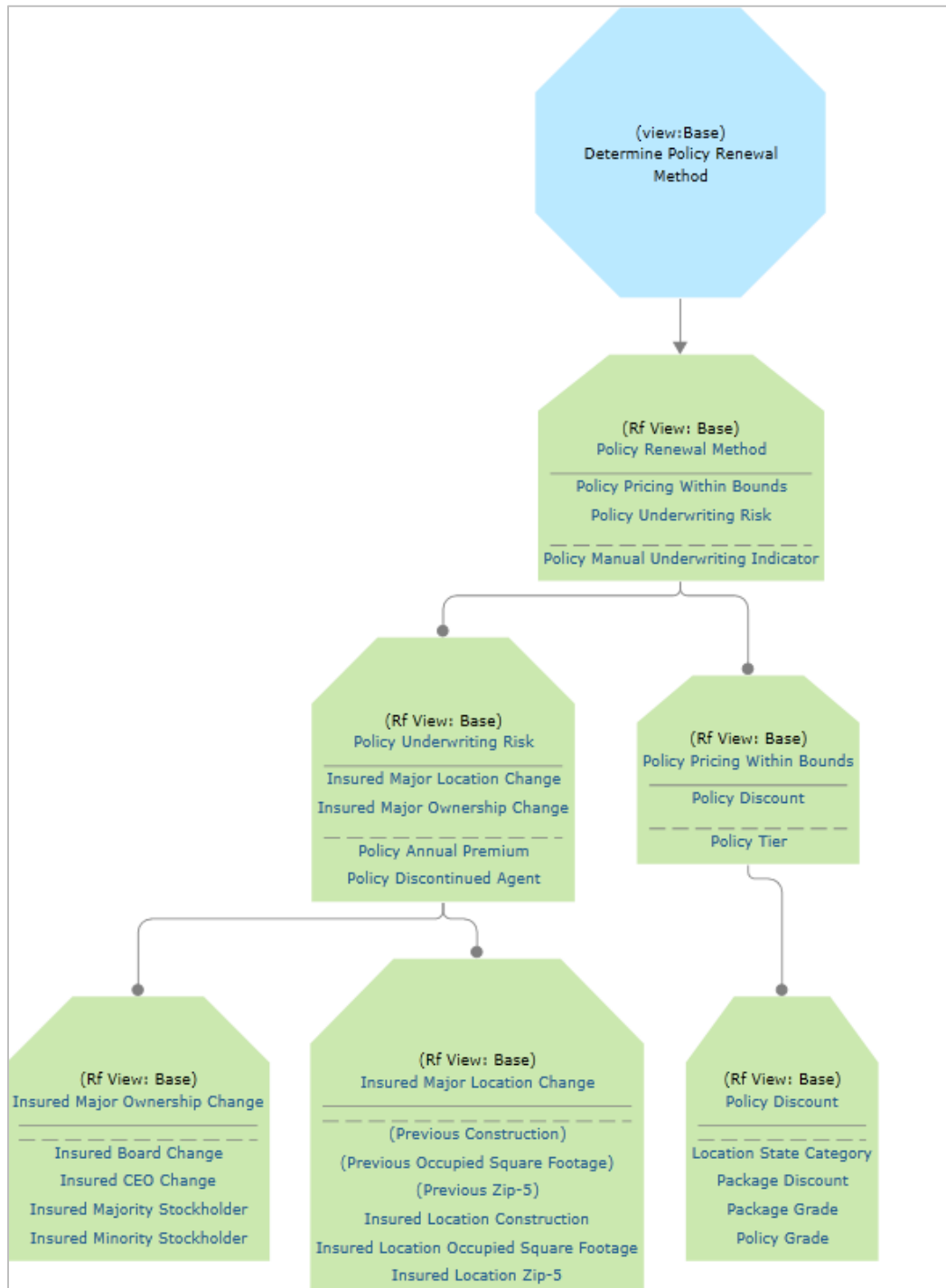
Within each RFV the questions – the FT's – are not ordered; any order is acceptable. Within the DV, MIO starts processing the Decision RFV and descend the hierarchy, one branch at a time until the branch is complete; then descend the hierarchy of the next branch. There is no order to the branches, any branch can be selected based on the least row ID of the RFV if there is a tie between the rows. Once the branch is selected, that hierarchy must be completed before the next hierarchy is chosen. This remains true for every level of every hierarchy.

The general method is that as question is answered, MIO executes that RFV to establish whether the answer given renders the RFV as determinative; if it is then, application executes the DV to determine whether it is determinative (the exception to this is the last question answered. This leads to a determinative DV, ending the process).

If the RFV is not determinative, application proceeds to the next FT in the RFV, present it on the page as the question to be answered, or if there are no further FTs in that RFV, move on to the next RFV in the hierarchy, or if at the end of a hierarchy (i.e., branch to a level the DV), to a new Hierarchy.

If the RFV is determinative, but not the DV, it means MIO completed processing the RFV and moves to the next RFV; if the DV is determinative, application completes the questionnaire.

Consider "Determine Policy Renewal Method" as an example:



In the below Decision Rule Family View, there are four rows to attain a conclusion, but out of which, three rows have the least number of Fact Types (shortest route) to reach a conclusion. In this case, there is a tie between the rows and application will fetch the least row ID. As per the example, "Policy Renewal Method" Fact Type has the least row ID, hence MIO fetches this Fact Type first.

SAPIENS Decision

Home Tasks (41) Repository Reports Administration

Loan Sanction Process > Policy Renewal Method (View: Base) [V1.1] > Policy Renewal Method (View: Base) [V1]

DRAFT NOT LATEST

	Policy Manual Underwriting Indicator	Policy Tier Within Bounds	Policy Underwriting Risk	Policy Renewal Method	Informational
1	Is : On	-	-	Is : Manual Renewal Process	"Policy Manual Underwriting Indicator"
2	-	Is : No	-	Is : Manual Renewal Process	"Policy Tier out of bounds"
3	-	-	Is : Unacceptable	Is : Manual Renewal Process	"Policy Underwriting Risk is unacceptable"
4	Is : Off	Is : Yes	Is : Acceptable	Is : Automatic Renewal Process	-

Additional Info

If the answer is "On", it is determinative for the decision; the Renewal Method is "Manual Renewal Method", and the questionnaire is done.

If the answer is "Off" and the DV is not determinative, then next question should be asked.

SAPIENS Decision

Home Tasks (41) Repository Reports Administration

Loan Sanction Process > Policy Renewal Method (View: Base) [V1.1] > Policy Tier Within Bounds (View: Base) [V1]

DRAFT

	Policy Discount	Policy Tier	Policy Tier Within Bounds
1	-	Is Less Than or Equal To : 1	Is : No
2	Is Less Than or Equal To : 10%	Is Greater Than : 1	Is : Yes
3	Is Greater Than : 10%	Is Less Than or Equal To : 1.5	Is : No
4	Is Less Than or Equal To : 20%	Is Greater Than : 1.5	Is : Yes
5	Is Greater Than : 20%	Is Less Than or Equal To : 2	Is : No
6	Is Less Than or Equal To : 25%	Is Greater Than : 2	Is : Yes
7	Is Greater Than : 25%	Is Less Than or Equal To : 2.6	Is : No
8	-	Is Greater Than : 2.6	Is : Yes

Based on the least row ID, MIO moves to the next lower branch, "Policy Pricing Within Bounds". (There are two branches at the next level, application chooses the shortest branch; it may have another of equal length in the other branch of the decision, but MIO evaluates one branch at a time. "Policy Pricing Within Bounds" branch has only a single interim fact type - the other branch has two. In any event, an arbitrary choice is fine for the method.)

In this RFV Policy the persistent FT is "Policy Tier". It is determinative for the DV when it's value > 2.6 (Policy pricing is within bounds regardless of Policy Discount), and that would mean for that value or greater in the answer, application keeps drilling down to the lower rule family and move to the next Branch.

If “Policy Tier” is less than or equal to 1, this is determinative for the RFV executes the DV, which is also determinative (sets “Policy Renewal Method” to “Manual”).

If “Policy Tier” is greater than 1, and less than 2.6, application evaluates "Policy Discount"; this is an interim FT, for which reason, application will evaluate the Policy Discount RFV.

SAPIENS Decision Home Tasks (41) Repository Reports Administration						
Loan Sanction Process > Policy Renewal Method (View: Base) [V1.1] > Policy Discount (View: Base) [V1]						
DRAFT						
	Location State Category	Package Discount	Package Grade	Policy Grade		Policy Discount
1	-	-	-	Is : D		Is : 0%
2	-	-	Is : B	Is In : {B,C}		Is : 5%
3	-	-	Is : A	Is In : {B,C}		Is : 10%
4	-	-	Is : B	Is : A		Is : 15%
5	Is In : {B,C,D,E,F}	-	Is : A	Is : A		Is : 20%
6	-	Is Greater Than or Equal To : 5%	Is : A	Is : A		Is : 20%
7	Is : A	Is Less Than : 5%	Is : A	Is : A		Is : 25%

Therefore, moving to the supporting “Policy Discount” Rule Family for the inputs to those persistent FTs, a user must input the values for the required fact types, and evaluate Policy Discount. This means that at this point you must add those FTs to the page. As the questions are answered, the RFV is evaluated.

Once “Policy discount” becomes determinative, and unless this renders the DV determinative, ending the questionnaire, application moves to the next branch.

5. Known Limitations

Following are the limitations identified during the current release.

- The MIO API does not consider Fact Types that used in messages as part of the algorithm. So, it is possible to get a conclusion with messages that have Fact Types that are not populated resulting in messages with null values.

Workaround: Modify the MIO API algorithm to include the Fact Types as required Fact Types. Any Fact Type used in a Message Category as a Fact Type or Cell Value will be considered as Fact Type annotation.

- MIO currently does not support Repeating group functionality.
- MIO currently does not consider Persistent FTs for Gateways as part of its algorithm.



Contact Us

For more information, please visit or contact us at:

info.sapiens@sapiens.com

For Support related queries, contact us at:

Decision.Support@sapiens.com

For Documentation related queries, contact us at:

Decision_documentation@sapiens.com

© 2022 Copyright Sapiens International. All Rights Reserved.

www.sapiensdecision.com



Partnering for Success